

1.11

файлы с кодом можно посмотреть [тут](#)

Условие

Используя любые источники информации, изучите метод обратной квадратичной интерполяции (ОКИ) и вычислите его порядок сходимости (собрав всю теоретическую информацию в отчет).

Решите с помощью этого метода уравнения

$$e^{-x} = x$$

и

$$x \sin(x) = 1$$

Для каждого уравнения постройте совмещенные диаграммы сходимости метода ОКИ, секущих и Ньютона, подтверждающие полученное значения порядка сходимости

```
import numpy as np
import matplotlib.pyplot as plt

# f1(x) = e^-x - x
def f1(x):
    return np.exp(-x) - x

# f1'(x) = -e^-x - 1
def df1(x):
    return -np.exp(-x) - 1

# f2(x) = x * sin(x) - 1
def f2(x):
    return x * np.sin(x) - 1

# f2'(x) = sin(x) + x * cos(x)
def df2(x):
    return np.sin(x) + x * np.cos(x)
```

Метод Ньютона

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \quad k = 0, 1, \dots \quad (4.5)$$

```

def newton_method(
    function, # функция
    derivative, # ее производная
    x0, # начальное приближение x0
    x_solution, # точное значение корня, чтобы считать ошибку
    eps=1e-12, # точность
    max_iterations=100 # max число итераций
):
    # print("Ньютон")
    # print(f"f: {function}")
    iteration_errors = [abs(x0 - x_solution)] # для хранения
    погрешностей на каждой итерации

    current_x = x0

    for k in range(max_iterations):
        # вычисляем значения
        f_value = function(current_x) # f(x_k)
        df_value = derivative(current_x) # f'(x_k)

        # далее мы будем на нее делить, поэтому проверим
        if df_value == 0:
            print("ERROR: Производная равна нулю")
            break

        # формула 4.5
        current_x = current_x - f_value / df_value
        # print(f"it: {k + 1}, current_x: {current_x + 1}")
        # добавляем ошибку |x_k - x*|
        error = abs(current_x - x_solution)
        iteration_errors.append(error)

        # f(x_k) < eps
        if abs(f_value) < eps:
            break
    # print()
    return iteration_errors

```

Метод секущих

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}, \quad k = 0, 1, \dots \quad (4.16)$$

```

def secant_method(
    f, # функция
    x0, x1, # начальные приближения
    x_solution, # точное значение корня, чтобы считать ошибку
    eps=1e-12, # точность

```

```

max_iterations=20 # макс число итераций
):
    # print("Текущая")
    # print(f"f: {f}")
    errors=[abs(x0 - x_solution), abs(x1 - x_solution)] # для
    хранения погрешностей
    a, b = x0, x1 # две текущие точки (x_{k-1}, x_k)

    for iteration in range(max_iterations):
        fa, fb = f(a), f(b) # значения функций в точках
        if fb - fa == 0: # далее будем на нее делить, надо
            удостовериться
            break

        x_new = b - fb * (b - a) / (fb - fa) # формула 4.16

        errors.append(abs(x_new - x_solution)) # добавляем ошибку
        if abs(f(x_new)) < eps: # f(x_k) < eps
            break
        a, b = b, x_new # x_{k-1}, x_k -> x_k, x_{k+1}
        # print(f"it: {iteration + 1}, a: {a}, b: {b}")
    # print()
    return errors

```

Метод обратной квадратичной интерполяции (ОКИ)

Используя любые источники информации, изучите метод обратной квадратичной интерполяции (ОКИ) и вычислите его порядок сходимости (составив всю теоретическую информацию в отчет).

в качестве источника я взял статью с Википедии ([ссылка](#)) и теорию с электронных конспектов ([тут](#) и [тут](#))

необходимая информация

$$\begin{aligned}
 x_{n+1} = & \frac{f_{n-1}f_n}{(f_{n-2}-f_{n-1})(f_{n-2}-f_n)}x_{n-2} + \frac{f_{n-2}f_n}{(f_{n-1}-f_{n-2})(f_{n-1}-f_n)}x_{n-1} \\
 & + \frac{f_{n-2}f_{n-1}}{(f_n-f_{n-2})(f_n-f_{n-1})}x_n,
 \end{aligned}$$

where $f_k = f(x_k)$. As can be seen from the recurrence relation, this method requires three initial values, x_0 , x_1 and x_2 .

Определение 4.3. Если при всех достаточно больших k имеет место неравенство

$$|\varepsilon_{k+1}| \leq C|\varepsilon_k^p|,$$

где $C < \infty$, $p \in \mathbb{R}$, то говорят, что метод, по которому построена последовательность (x_k) , имеет *порядок (скорость) сходимости* p . Если $p = 1$, то сходимость называют *линейной*, при $1 < p < 2$ — *сверхлинейной*, $p = 2$ — *квадратичной* и т. д.

Теорема 2.3. Пусть $f \in C^{(n+1)}[a, b]$, P_n — ИМ для f по узлам $\{x_i\}_{i=0}^n \subset [a, b]$. Тогда $\forall x \in [a, b] \exists \xi \in (\underline{x}, \bar{x})$, $(\underline{x} = \min\{x_0, \dots, x_n, x\}, \bar{x} = \max\{x_0, \dots, x_n, x\})$, такое, что

$$r_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x). \quad (2.14)$$

Доказательство. Данная теорема является тривиальным следствием теоремы 2.1 и леммы 2.2. ■

Формула (2.14) является основным инструментом при оценке погрешности интерполяции. Из неё, в частности, сразу получаем

$$\varepsilon_n = \|r_n\| \leq \frac{\|f^{(n+1)}\|}{(n+1)!} \|\omega_{n+1}\|, \quad (2.15)$$

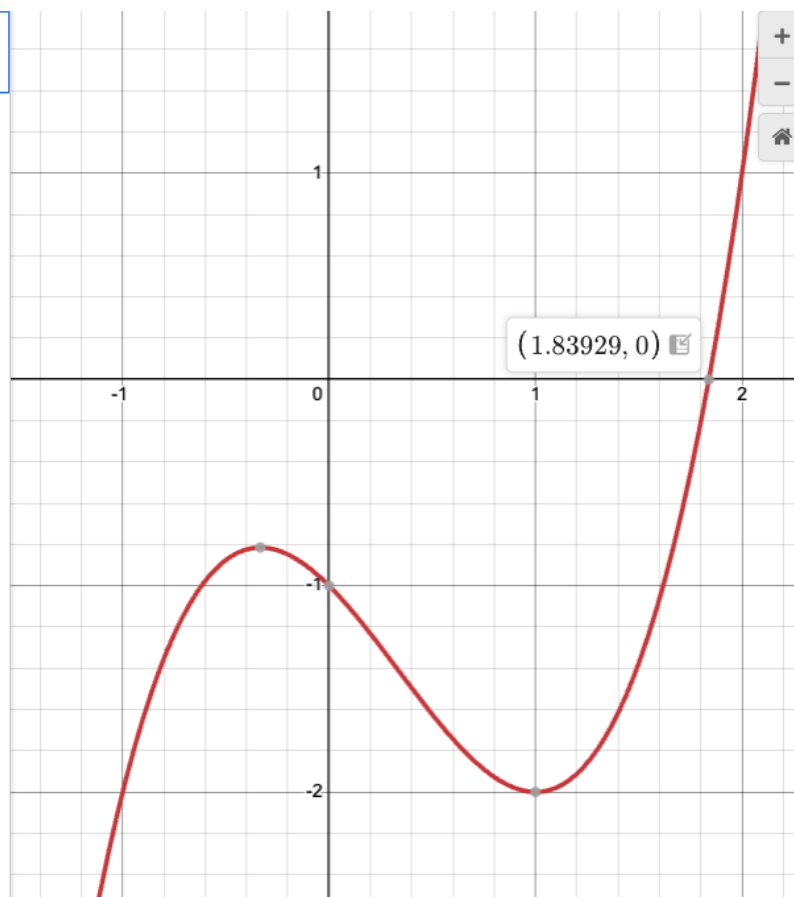
где $\|\cdot\| = \|\cdot\|_{C[a,b]}$ (см. (2.11)).

Таким образом получаем знаменитую формулу интерполяционного многочлена в форме Лагранжа:

$$P_n(x) = \sum_{i=0}^n y_i \Lambda_i(x) = \sum_{i=0}^n y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}. \quad (1.8)$$

1
2


$$y = x^3 - x^2 - x - 1$$

Вычисление порядка сходимости

$f(x)$ — непрерывная функция
 x^* — корень $f(x)$ $[f(x^*) = 0]$
 $g(y) = f^{-1}(y)$ — обратная ф-ция
 $[x^* = g(0)]$.

Формула итерационного процесса:

$$\begin{aligned}
 x_{n+1} = & \frac{f_{n-1} f_n}{(f_{n-2} - f_{n-1})(f_{n-2} - f_n)} x_{n-2} + \\
 & + \frac{f_{n-2} f_n}{(f_{n-1} - f_{n-2})(f_{n-1} - f_n)} x_{n-1} + \\
 & + \frac{f_{n-2} f_{n-1}}{(f_n - f_{n-2})(f_n - f_{n-1})} x_n
 \end{aligned}$$

где $f_k = f(x_k)$

Далее: $y_k = f(x_k)$.

Построим Л.М. в форме Лагранжа
 по n -м $y_k = f(x_k)$, $y_{k-1} = f(x_{k-1})$,
 $y_{k-2} = f(x_{k-2})$

$$\left[P_n(x) = \sum_{i=0}^n y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}, \text{ где } \begin{matrix} x \rightarrow x_j \\ y \rightarrow y_i \end{matrix} \right]$$

$$\begin{aligned}
 P_3(y) = & \frac{(y - y_{k-1})(y - y_{k-2})}{(y_k - y_{k-1})(y_k - y_{k-2})} x_k + \\
 & + \frac{(y - y_k)(y - y_{k-2})}{(y_{k-1} - y_k)(y_{k-1} - y_{k-2})} x_{k-1} + \frac{(y - y_k)(y - y_{k-1}) \cdot x_{k-2}}{(y_{k-2} - y_k)(y_{k-2} - y_{k-1})}
 \end{aligned}$$

т.о. $x_{k+1} = P_2(0)$

Далее согласно теореме 2.3 из лекционного конспекта, имеем:

$$\begin{aligned} z_2(y) &= g(y) - P_2(y) = \\ &= \frac{g^{(3)}(\xi)}{3!} \omega_3(y) = \text{const} \cdot \omega_3(y) \end{aligned}$$

$$z_2(0) = \frac{\text{const}}{C} \cdot y_k y_{k-1} y_{k-2}$$

$$g(0) - P_2(0) = x^* - x_{k+1} = e_{k+1}$$

т.о. $e_{k+1} \in C y_k y_{k-1} y_{k-2}$

Далее, выразим y_k через e_k

Разложим $f(x)$ в ряд Тейлора в окрестности x^* и получим значение $f(x_{k+1})$:

$$\begin{aligned} f(x_{k+1}) &= f(x^*) + f'(x^*)(x_{k+1} - x^*) + \\ &+ \frac{f^{(2)}(x^*)}{2!} (x_{k+1} - x^*)^2 + \dots = \end{aligned}$$

$$\Rightarrow \left[x_k \rightarrow x^*, f(x^*) = 0 \right] \Rightarrow f'(x^*) e_{k+1}$$

$$\text{и тогда } f(x_{k+1}) = f'(x^*) e_{k+1} = y_{k+1}$$

с учетом того, что $f'(x^*) = \text{const}$, получаем:

$$e_{k+1} = C e_k \cdot e_{k-1} e_{k-2}$$

$$|e_{k+1}| \leq C |e_k| \cdot |e_{k-1}| \cdot |e_{k-2}|$$

из определения инверсной квадратичности:

$$|e_{k+1}| \leq C |e_k|^p$$

Далее мы рассмотрим:

$$\begin{cases} |e_{k+1}| = A |e_k|^p, [A = \text{const}] \\ |e_{k+1}| = C |e_k| \cdot |e_{k-1}| \cdot |e_{k-2}| \end{cases}$$

$$|e_k| = A |e_{k-1}|^p \Rightarrow |e_{k-1}| = A^{-1/p} |e_k|^{1/p}$$

$$|e_{k-2}| = A^{-1/p} |e_{k-1}|^{1/p} =$$

$$= A^{-1/p - 1/p^2} |e_k|^{1/p^2}$$

С учетом констант подставим:

$$|e_k|^p = |e_k|^1 \cdot |e_k|^{1/p} \cdot |e_k|^{1/p+2}$$

чтобы при $|e_k| \rightarrow 0$ воп-ое асимптотическое равенство, нужно чтобы воп.!

$$p = 1 + \frac{1}{p} + \frac{1}{p^2}$$

$$p^3 - p^2 - p - 1 = 0, \text{ у которого ед.-ой}$$

корень $p \approx 1.84.$

```
def inverse_quadratic_interpolation_method(
    f, # функция
    x0, x1, x2, # начальные приближения
    x_solution, # точное значение корня, чтобы считать ошибку
    eps=1e-12, # точность
    max_iterations=20 # макс число итераций
):
    # print("ОКИ")
    # print(f"function: {f}")
    errors=[abs(x0 - x_solution), abs(x1 - x_solution), abs(x2 -
x_solution)] # для хранения погрешностей
```

```

a, b, c = x0, x1, x2 # три текущие точки (x_k-2, x_k-1, x_k)

for iteration in range(max_iterations):
    fa, fb, fc = f(a), f(b), f(c) # значения функций в точках

    if fa == fb or fa == fc or fb == fc: # чтобы в дальнейшем не
поделить на 0
        break

    # применяем формулу
    term1 = (fb * fc) / ((fa - fb) * (fa - fc)) * a
    term2 = (fa * fc) / ((fb - fa) * (fb - fc)) * b
    term3 = (fa * fb) / ((fc - fa) * (fc - fb)) * c
    x_new = term1 + term2 + term3

    errors.append(abs(x_new - x_solution)) # добавляем ошибку
    if abs(f(x_new)) < eps: # f(x_k+1) < eps
        break
    a, b, c = b, c, x_new # x_k-2, x_k-1, x_k -> x_k-1, x_k, x_k+1
    # print(f"it: {iteration + 1} a: {a}, b: {b}, c: {c}")
    # print()
    return errors

```

Вычисления и графики

порядки сходимостей методов Ньютона и Секущей 2 и 1.618 соответственно (исходя из)

```

# уравнение 1
x_solution_1 = 0.5671432904097838
err_newton_1 = newton_method(f1, df1, 0.5, x_solution_1)
err_secant_1 = secant_method(f1, 0.0, 1.0, x_solution_1)
err_iqi_1 = inverse_quadratic_interpolation_method(f1, 0.0, 1.0,
0.5, x_solution_1)

# уравнение 2
x_solution_2 = 1.1141571408719300
err_newton_2 = newton_method(f2, df2, 1.5, x_solution_2)
err_secant_2 = secant_method(f2, 1.5, 1.4, x_solution_2)
err_iqi_2 = inverse_quadratic_interpolation_method(f2, 1.4,
1.1456745414688143, 1.1159043790663805, x_solution_2) #
1.0163475399040172, 1.1050056016809153, 1.1140348254936252

# функция для построения
def plot_convergence(err_newton, err_secant, err_iqi, title):
    plt.figure(figsize=(8, 6))
    plt.plot(range(len(err_newton)), err_newton, 'o-', label='Ньютона
(p=2)')
    plt.plot(range(len(err_iqi)), err_iqi, 's-', label='ОКИ (p≈1.84)')
    plt.plot(range(len(err_secant)), err_secant, '^-', label='секущих

```

```
(p≈1.618)')
```

```
plt.yscale('log')
plt.xlabel('итерация, k')
plt.ylabel('$|x_{k} - x^{*}|$')
plt.title(title)
plt.grid(True, which="both", ls="--", alpha=0.7)
plt.legend()
plt.tight_layout()
plt.show()
```

```
# построение
```

```
plot_convergence(err_newton_1, err_secant_1, err_iqi_1, '$e^{-x} - x = 0$')
plot_convergence(err_newton_2, err_secant_2, err_iqi_2, '$x \sin x - 1 = 0$')
```

